The chapter question

Between `sum += a[i]` and transistor switching lie several contracts. This chapter gives each contract a boundary that can be described and measured.

MIPS32 is the teaching instruction set architecture, or ISA. Its use does not imply that current processors all execute MIPS. ARM and x86 enter as comparative cases.

Ask which layer determines whether multiplication is an instruction, then which layer determines its cycle count. The first question concerns the ISA. The second concerns its implementation.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Prerequisites and outcomes

The mathematical prerequisites are binary notation, scientific notation, ratios, and weighted averages. You should be able to predict execution time with consistent units and state the conditions under which the prediction fails.

Every performance claim needs a workload. A processor being "fast" without an input, operation, and measurement method is an incomplete statement.

Repair a processor advertisement by supplying the workload, time unit, and method that would make its claim testable.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

1.1 Computer abstraction layers

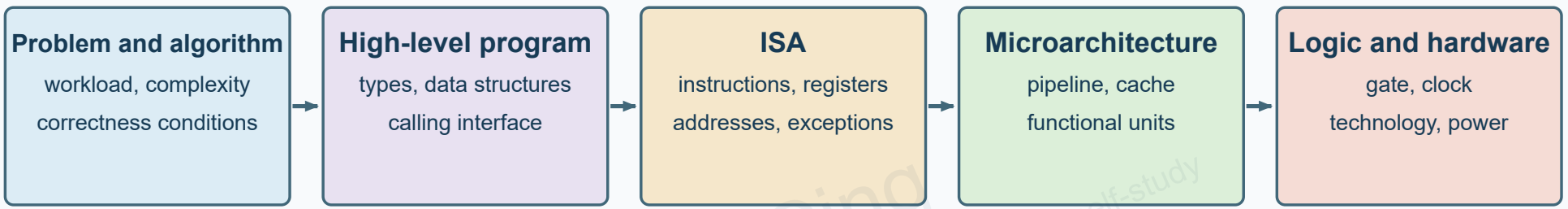
Applications use libraries and system calls. Compilers generate ISA instructions. Microarchitectures implement these instructions as datapath operations. Logic and physical technology implement registers, adders, and memory.

An interface specifies observable behavior, so the upper layer need not describe every transistor. Information still crosses layers during optimization. Data locality, for example, affects both compiler transformations and cache behavior.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

1.1 Computer abstraction layers (continued)

Interfaces connect abstraction layers



Upper layers define observable contracts; lower layers implement them. Cross-layer optimization preserves these interfaces.

An abstraction does not conceal everything. Instructions, addresses, exceptions, and performance counters expose selected lower-level behavior.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

ISA and microarchitecture

The ISA defines software-visible instructions, registers, data types, addressing, privilege, and exceptions. Processors with the same ISA may have different pipeline depths, caches, and execution units.

The microarchitecture defines how that contract is implemented. It determines the work in each clock cycle and the number of instructions that can remain in flight.

Prediction, caching, and reordering are valid only when committed behavior satisfies the architectural contract. Memory ordering rules constrain observations that concurrent software may make.

The same function interface can admit different internal algorithms. Interface compatibility alone says nothing about their costs.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

One ISA, two implementations: timing three instructions



Assume the same executable and input, three independent instructions, correct ISA implementations, no stalls or exceptions, and a MEM stage even for non-memory instructions. The single-cycle design uses one 800 ps cycle per instruction, finishing in $3 \times 800 = 2400$ ps.

One ISA, two implementations: timing three instructions (continued)

The five-stage design uses 200 ps cycles and takes seven cycles from the first IF through the final WB: $7 \times 200 = 1400$ ps. Its speedup is $2400/1400 = 12/7$. The same instruction results do not imply the same execution time.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Application classes and design tradeoffs

Personal computers, servers, embedded controllers, and mobile devices face different constraints. Servers often value throughput and availability. Mobile systems have battery and thermal limits. Embedded control may require a bounded worst-case response time.

A larger cache can improve a workload while consuming area and leakage power. Deep sleep can save energy while increasing wake-up latency. A useful design comparison includes performance, cost, power, dependability, and programmability.

Rank the constraints for a data-center CPU and a pacemaker. The appropriate optimization objective changes with the application.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Course map

Chapter 2 establishes the ISA. Chapter 3 develops integer and floating-point arithmetic. Chapter 4 builds a datapath and pipeline. Chapter 5 introduces caches and virtual memory. Chapter 6 covers storage and I/O. Chapter 7 connects these ideas to multicore processors, multiprocessors, and clusters.

Dimensional analysis, comparisons against an explicit baseline, and workload-based validation recur throughout. For every new mechanism, identify the resource and time component it changes.

Thread	Initial model	Mechanisms developed later
Data	Operands, results, and instruction count	Registers/addresses (2) → arithmetic (3) → datapath (4) → cache (5)
Control	ISA state and execution time	Branch/call (2) → hazards/exceptions (4) → I/O completion (6)
Communication	Bytes moved and shared resources	Coherence (5) → DMA/links (6) → synchronization/networks (7)

1.2 From a program to hardware

A compiler translates a high-level language into assembly. An assembler encodes instructions. A static linker combines object files and resolves symbols and relocations. A loader establishes the process address space. The processor decodes and executes instructions according to the ISA.

Each stage leaves inspectable evidence. Compilation can change instruction count and register allocation. Assembly determines encodings. Linking determines final symbol references. Loading establishes initial mappings.

Source-code line count does not measure executed work. Optimization settings, data types, and compiler output can all change the dynamic instruction count.

Distinguish source, assembly, and machine code. The ISA connects symbolic instructions to their architectural effects and binary encodings.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Interfaces and portability

An executable can run on implementations of its ISA only if its required architectural features are supported. Full binary compatibility also requires the operating-system interface and application binary interface, or ABI.

The ABI specifies register preservation, calling conventions, data alignment, and executable formats. These rules connect independently compiled code to the toolchain and runtime.

A processor can maintain an older ISA while translating its instructions into internal micro-operations. This is an implementation choice. Compatibility, portability, and equal performance are separate claims.

Ask which contract fails when code has the right instruction set but the wrong system-call interface.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

1.3 Under the covers

A system contains processors, volatile memory, nonvolatile storage, I/O devices, and interconnects. Input devices turn external state into data. Output devices turn data into observable effects.

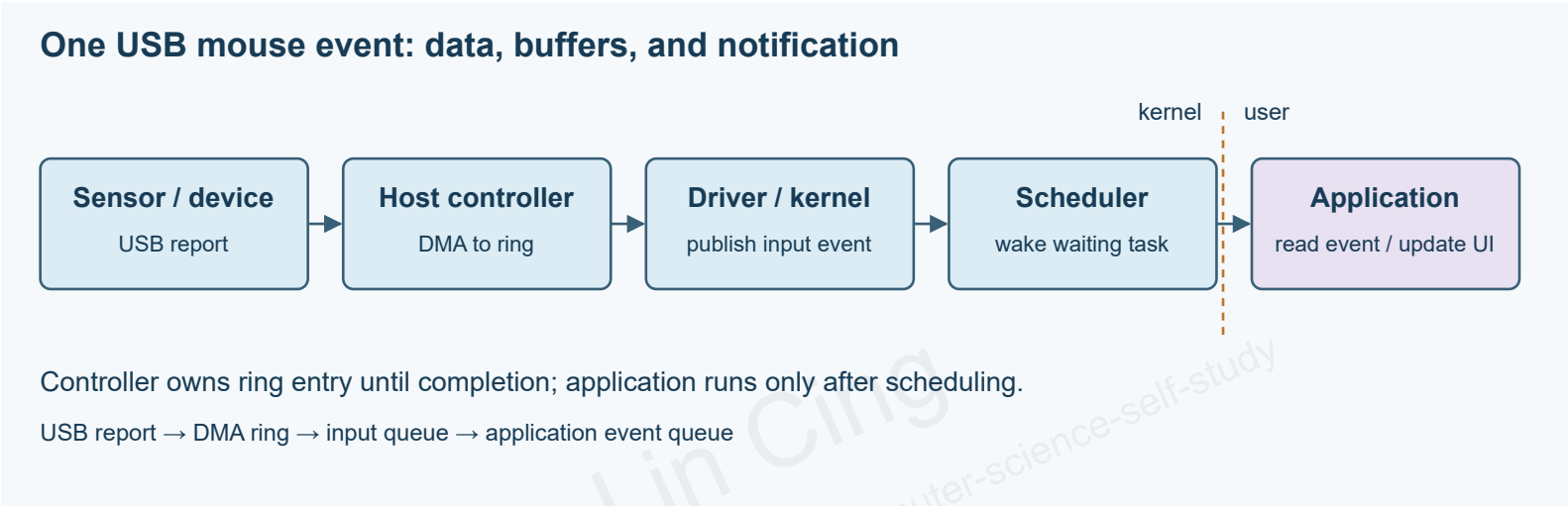
The processor contains a datapath and control. DRAM and persistent storage lie outside the core. Shared buses or point-to-point links carry addresses, data, and control messages.

Peripherals affect system performance through interrupts, direct memory access, and memory-mapped I/O. Their behavior belongs in an end-to-end model.

Fix a USB mouse, a host-controller DMA ring, and an application waiting for input. Buffers below identify separate ownership boundaries; an arrow does not imply that the entire report is copied.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

The path of a USB input event



The controller writes a report to a driver-managed ring. Completion notification lets the driver consume the finished entry and publish an input event. The scheduler then wakes the application, which reads its event queue. A ring slot cannot be reclaimed before DMA completion; application wake-up is later than packet arrival.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A mouse as an interface example

A sensor produces motion information. A device controller packages that information and sends it through an interface such as USB or Bluetooth to a host controller. The driver passes events into the operating system's input subsystem.

Events may be batched instead of waking the application for every sample. Batching trades CPU wake-up energy against latency.

Slow visible response can originate in sampling, transport, scheduling, the application's frame rate, or display refresh. A single component rate cannot identify the cause.

Decompose end-to-end latency before assigning blame to the processor.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A tablet as a system example

A system on chip, or SoC, can integrate CPU cores, a GPU, media processors, a memory controller, and I/O controllers. Integration reduces packaging and communication costs but concentrates heat and power density.

Touch response includes sampling, computation, GUI composition, and display scan-out. Completing a CPU burst earlier may still leave a display-stage wait.

Short bursts and long idle intervals make averages incomplete. Tail latency and energy spent changing power states can determine the user experience.

Identify five components that can compete for the same memory bandwidth.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Motherboards and datapaths

The motherboard provides physical connections among chips and peripherals. Point-to-point links avoid some of the contention and electrical loading of a single shared bus.

On-chip interconnects connect cores, last-level caches, and memory controllers. Cache-coherence protocols regulate the visibility of copies of shared data.

Topology affects hop counts and contention. A link's peak bandwidth is an upper bound on traffic through that link, not a promise of sustained application bandwidth.

Describe an interconnect using nodes, links, routers, and a protocol.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Where data survives

Registers, caches, and DRAM are volatile. Flash, magnetic disks, and optical media retain data without continuous power.

Reliability, backup, and crash consistency address different failures. RAID can tolerate selected device failures. It does not replace an independent backup. A mean time between failures does not prove that a file system recovers a consistent state after a crash.

A cache is a temporary copy. A dirty line may hold the newest value before memory is updated. The logical data model must remain valid despite these changes in physical location.

A backup can exist yet take too long to restore. Recovery time is a separate requirement.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Networks in system organization

Network latency, bandwidth, and protocol overhead affect both performance and availability. Propagation distance, switch queues, and congestion control contribute different delays.

An object may move from storage into memory and then reach the network interface through DMA. The CPU need not copy each byte, but it must arrange descriptors and handle completion.

Compute, storage, and communication must be modeled together. Fast arithmetic does not remove an I/O wait.

In a serial teaching model with an established connection, take encoding as 0.2 ms, queueing as 0.3 ms, the outbound transmission/propagation as 2.0 ms, server computation as 1.5 ms, and the return path as 2.0 ms. Latency is 0.2+0.3+2.0+1.5+2.0=6.0 ms. Accelerating only server computation by three gives 5.0 ms and speedup 6/5=1.2. These are declared model inputs. Overlapping communication and computation requires a dependency critical-path calculation instead of this serial sum.

Technology and cost

Integration density and device dimensions constrain chip design. Smaller devices can reduce area and switching energy per function, but leakage, variability, and fabrication costs prevent automatic cost reductions.

The cost of a usable die is not simply proportional to die area. Larger dies fit fewer times on a wafer and expose a larger area to defects. Yield therefore matters as well as raw area.

Specify the cost boundary. Die fabrication, packaging, power delivery, cooling, racks, and operating energy have different denominators.

Additional transistors improve a program only when an architectural or parallel mechanism makes useful use of them.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Die area, yield, and cost

Fix wafer diameter $D = 200$ mm, wafer cost 1000 cost units, and die area A in mm². Use the geometric edge-loss approximation

$$N_{\text{die}} = \left\lfloor \frac{\pi (D/2)^2}{A} - \frac{\pi D}{\sqrt{2A}} \right\rfloor .$$

Assume negative-binomial yield $Y = (1 + dA_c/\alpha)^{-\alpha}$, $d = 0.5$ defects/cm², $\alpha = 2$, and $A_c = A/100$ cm² (100 mm²=1 cm²). Expected good-die cost is $1000/(N_{\text{die}} Y)$, excluding packaging, test, and redundancy.

Die area	Area term minus edge term	Gross dies	Yield	Expected good dies	Cost/good die
100 mm²	floor(314.159−44.429)	269	(1+0.5/2)$^{-2}$=0.64	172.16	5.8086
200 mm²	floor(157.080−31.416)	125	(1+1/2)$^{-2}$=4/9	55.5556	18.0000

Doubling area lowers count and yield: cost rises 18/5.8086≈3.10 times. Good-die count is an expectation; actual output is integral. All inputs are teaching assumptions, not manufacturing prices.

1.4 Defining performance

Response time is the elapsed time between the start and completion of a task. Throughput is completed work per unit time.

Shortening a critical path may reduce response time. Pipelining, batching, or adding cores may primarily increase throughput.

For a common workload and baseline,

$$S = \frac{T_{\text{old}}}{T_{\text{new}}}, \qquad T_{\text{new}} = \frac{T_{\text{old}}}{S}.$$

Halving time gives speedup 2 and a 50% time reduction. These are different ways of reporting the same comparison.

A claim of twice as many requests per second describes throughput. It does not specify each request's latency.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Latency and throughput

In a five-station pipeline, one task's latency includes all five stages. After filling, a pipeline may complete one task per stage interval. Reducing the completion interval need not reduce individual latency proportionally.

Batching amortizes setup work but may delay the first result. Cores can process separate tasks concurrently but still compete for shared resources and pay synchronization costs.

Fix the workload, input size, warm-up, software version, and observation window before comparing systems.

Decide whether twice the throughput justifies 10% worse latency for a specific application.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Benchmarks, ratios, and the geometric mean

A benchmark must specify its workload, input size, system configuration, compiler flags, warm-up, and measurement window. Scores obtained under different conditions describe different experiments.

For program i, divide the reference time by the measured time:

$$r_{X,i} = \frac{T_{\text{ref},i}}{T_{X,i}}, \qquad GM_X = \left(\prod_{i=1}^{n} r_{X,i} \right)^{1/n}.$$

Larger ratios mean shorter measured times. The reason for combining ratios geometrically follows directly:

$$\frac{GM_X}{GM_Y} = \left(\prod_{i=1}^{n} \frac{r_{X,i}}{r_{Y,i}} \right)^{1/n} = \left(\prod_{i=1}^{n} \frac{T_{Y,i}}{T_{X,i}} \right)^{1/n}.$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Benchmarks, ratios, and the geometric mean (continued)

Each program's reference time cancels in the paired X/Y ratio. Changing the reference machine leaves this ratio of geometric means unchanged, even when the new reference times change by different factors. The arithmetic mean has no such invariance. A suite summary still does not represent every workload. If one kernel occupies 90% of a user's time, reweight that workload or compare the kernel directly.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Reconstructing CPU time

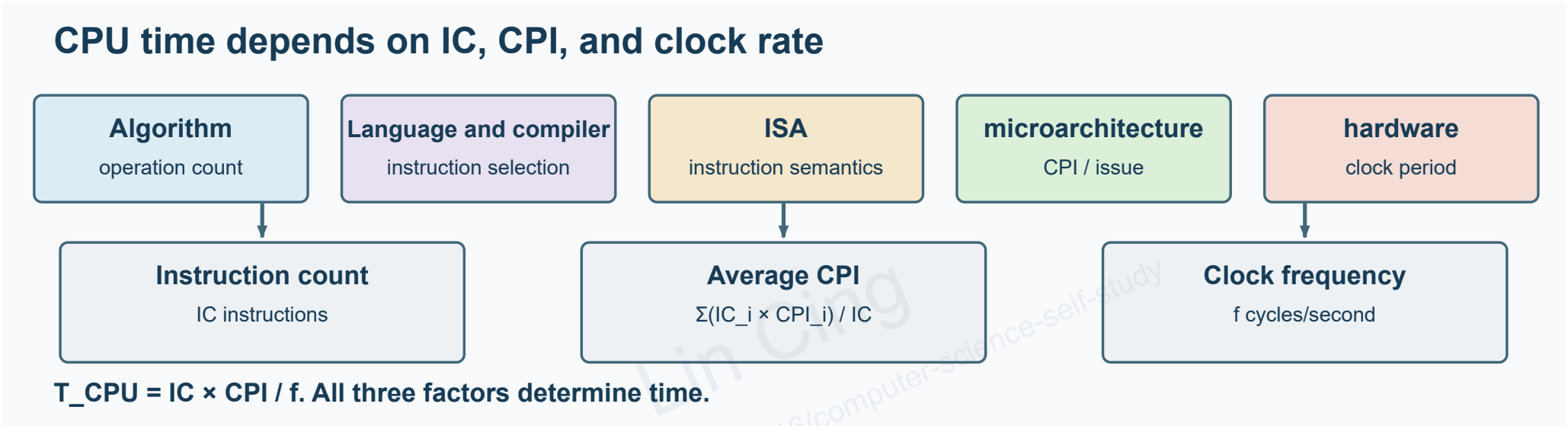
CPU time counts time spent executing for the program. For one thread over the same observation window, with disjoint user/system accounting,

$$T_{\text{elapsed}} \geq T_{\text{CPU}} = T_{\text{user}} + T_{\text{system}}.$$

I/O waits, descheduling, and OS work for other programs consume elapsed time. Summing thread CPU times can exceed elapsed time. Four cores each working for one second contribute 4 CPU-s during 1 elapsed s. State the thread/process accounting and time denominator.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Reconstructing CPU time (continued)



Begin with the cycle count and then multiply by seconds per cycle:

$$C = IC \times CPI, \qquad T_{\text{CPU}} = CT_c = IC \times CPI \times T_c = \frac{IC \times CPI}{f}.$$

Reconstructing CPU time (continued)

The dimensions cancel:

$$\text{instructions} \times \frac{\text{cycles}}{\text{instruction}} \times \frac{\text{seconds}}{\text{cycle}} = \text{seconds}.$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete comparison

The same program executes $I = 10^9$ ISA instructions on A and B. A has CPI 2.0 and a 250 ps clock period. B has CPI 1.2 and a 500 ps clock period.

$$C_A = 10^9(2.0) = 2.0 \times 10^9, \qquad T_A = 2.0 \times 10^9(250\,\text{ps}) = 0.500\,\text{s}.$$
$$C_B = 10^9(1.2) = 1.2 \times 10^9, \qquad T_B = 1.2 \times 10^9(500\,\text{ps}) = 0.600\,\text{s}.$$
$$S_{A/B} = T_B/T_A = 0.600/0.500 = 1.2.$$

A is 1.2 times as fast. B's lower CPI loses to its longer cycle. Comparing any single factor omits the other two. This example directly assumes equal dynamic IC. The same executable and input yield equal counts if they produce the same instruction trace; the same ISA and high-level program alone do not ensure equal counts. Compiler choices, settings, and execution environment may change the count.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Weighted CPI and the instruction-count trap

If class i executes IC_i instructions and CPI_i is an additive, non-overlapping allocation of total cycles per instruction,

$$C = \sum_i IC_i CPI_i, \qquad CPI_{\text{avg}} = \frac{\sum_i IC_i CPI_i}{\sum_i IC_i}.$$

The denominator counts all executed instructions, not classes or source lines. Pipelined or superscalar instructions can overlap, so individual execution latencies cannot simply be added. The following example specifies additive cycle costs. If frequency varies, sum time over intervals or use the effective average cycle rate C/T over the same active interval.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete counterexample

Instruction class	A	B	C
CPI	1	2	3
Sequence 1 count	2	1	2
Sequence 2 count	4	1	1

$$C_1 = 2(1) + 1(2) + 2(3) = 10, \qquad CPI_1 = 10/5 = 2.0.$$
$$C_2 = 4(1) + 1(2) + 1(3) = 9, \qquad CPI_2 = 9/6 = 1.5.$$

At the same frequency,

$$T_1 = 10T_c, \quad T_2 = 9T_c, \quad S_{2/1} = 10/9 \approx 1.111.$$

Sequence 2 executes one more instruction but takes one fewer cycle. Total cycles determine CPU time. A changed instruction mix invalidates an assumed constant CPI. Profilers, ISA simulators, and hardware counters can supply class counts and cycle observations; record each method's coverage and overhead.

Limits of the classic equation

The CPU-time equation is an accounting identity, not a guarantee that a local change improves total time. Its factors must describe the same execution.

CPU time differs from wall-clock response time. I/O waits, descheduling, and network delays may lie outside the measured CPU time.

A parallel model also needs the core count p, parallel fraction, imbalance, and synchronization overhead. Chapter 7 adds Amdahl's law and the Roofline model.

At fixed frequency, a lower CPI can still lose if the compiler increases instruction count enough.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete performance budget

Program A executes 2.0×10^8 instructions at CPI 1.25 and 2.0 GHz.

1. Cycles are $(2.0 \times 10^8)(1.25) = 2.5 \times 10^8$.
2. Cycle time is $1/(2.0 \times 10^9) = 0.5$ ns.
3. CPU time is $2.5 \times 10^8 (0.5\ \text{ns}) = 0.125$ s.

$$T_A = \frac{(2.0 \times 10^8)(1.25)}{2.0 \times 10^9} = 0.125\ \text{s}.$$

A new compiler produces 1.6×10^8 instructions, but CPI rises to 1.40 at the same frequency:

$$C_B = 1.6 \times 10^8 (1.40) = 2.24 \times 10^8,$$

$$T_B = \frac{2.24 \times 10^8}{2.0 \times 10^9} = 0.112\ \text{s}, \qquad S = 0.125/0.112 = 1.1161.$$

Instruction count falls 20%, while CPI rises 12%. The time saving is only

$$(0.125 - 0.112)/0.125 = 10.4\%.$$

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Complete performance budget (continued)

Incorrectly retaining A's CPI gives 0.100 s and speedup 1.25. That calculation ignores the changed instruction mix.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

1.5 The power wall

An approximate switching-power model is

$$P_{\text{dyn}} = \alpha C V^2 f,$$

where C is switched capacitance, V is supply voltage, f is switching frequency, and α is the activity factor. Raising frequency often requires higher voltage, so power can rise faster than frequency.

Leakage also contributes. Lowering supply voltage alone generally reduces leakage but slows circuits. Lowering threshold voltage to recover speed can increase subthreshold leakage and sensitivity to process variation.

Cooling constrains sustained power density. A processor may throttle or disable resources when thermal limits are reached. A thermal design rating should not be interpreted as an exact universal throttling threshold.

Identify the controlled quantities before reasoning about device scaling or core count.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

From one core to many and the derivation of Amdahl's law

Power density, clock distribution, limited instruction-level parallelism, and memory latency prevent indefinitely raising single-core frequency. Multiple cores increase aggregate throughput and can reduce elapsed time when work splits into independent parts.

A fixed workload takes baseline time T_0. A fraction fT_0 can be shared among N workers; $(1-f)T_0$ remains serial. Assume equal work and no added overhead:

$$T_N = (1-f)T_0 + \frac{fT_0}{N} = T_0\left(1 - f + \frac{f}{N}\right), \qquad S_N = \frac{T_0}{T_N} = \frac{1}{1 - f + f/N}.$$

As $N \to \infty$, the parallel term tends to zero but the serial term remains:

$$S_\infty = \frac{1}{1-f}.$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Required parallel fraction

For a target finite-N speedup S,

$$\frac{1}{S} = 1 - f + \frac{f}{N} = 1 - f\left(1 - \frac{1}{N}\right),$$

so

$$f \geq \frac{N(S-1)}{S(N-1)}, \qquad 1 < S < N.$$

With $N = 100, S = 90$,

$$f \geq \frac{100(89)}{90(99)} \approx 0.998878.$$

The serial fraction must be at most about 0.1122%. This ideal model cannot attain $S > N$. It attains $S = N$ when $f = 1$ and added overhead is zero. Strict $S_N < N$ for finite $N > 1$ requires $f < 1$.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

An unattainable finite target

A program takes 100 s, with multiplication using 80 s and other operations 20 s. A 5-fold overall speedup would require

$$20 = \frac{80}{s_{\text{mul}}} + 20.$$

Thus $80/s_{\text{mul}} = 0$, attainable only as a limit as $s_{\text{mul}} \to \infty$. No finite local speedup achieves it. For a 2-fold speedup,

$$50 = \frac{80}{s_{\text{mul}}} + 20 \quad \Rightarrow \quad s_{\text{mul}} = 80/30 = 2.667.$$

Measure the improved part's fraction of baseline elapsed time first. Operation-count and source-line fractions do not supply that time fraction.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Manufacturing and the AMD Opteron case

Fabrication, test, packaging, and system integration all contribute to delivery cost. Yield learning and design for test affect the count of usable chips. A configured machine's benchmark result alone does not establish manufacturing cost.

The book's AMD Opteron X4 example belongs to the late 2000s. Interpret it using its contemporary process, ISA, workload, and server configuration.

A reproducible benchmark needs at least processor configuration, memory, software versions, compiler settings, and input data.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

SPEC CPU measurements

Speed-oriented results describe program execution time. Rate-oriented results describe completed work under a specified concurrent configuration.

A normalized reference score is a ratio, not a number of seconds. Adding ratios does not produce total performance.

Keep each benchmark's ratio, hardware configuration, compiler flags, and inputs. A geometric mean summarizes the suite but does not replace those observations.

If two machines win different programs, the user's workload determines which result matters.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

SPEC power measurements

Power benchmarks relate completed work to power across load levels, including idle behavior. Peak performance alone can miss the cost of a server that spends much of its time at low utilization.

Power is measured in watts. Energy is measured in joules.

$$E = \int P(t)\,dt.$$

Compare energy for the same completed work. Account for idle power and state-transition costs when these occupy a material fraction of the observation window.

Determine whether a reported quantity is average power or accumulated energy before combining it with time.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

1.8 Fallacies and pitfalls

A local speedup is not the overall speedup. If a component occupies only 10% of baseline time, eliminating its time saves at most 10% overall.

A benchmark win is workload-specific. Database, compiler, and numerical workloads may stress different resources.

More resources can increase energy and management cost without increasing completed work when utilization or shared bottlenecks prevent useful scaling.

For each claim, construct a counterexample and identify the assumption it violates.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Abstractions and interfaces

Execution time is a joint result of the workload, ISA, compiler, microarchitecture, memory, and I/O. Use four questions throughout the course.

1. Is the measured quantity latency or throughput?
2. Which of instruction count, CPI, and frequency changed?
3. Which resources are shared, and are they saturated?
4. Can the benchmark and configuration be reproduced?

Apply these questions whenever a new optimization mechanism appears.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Historical perspective

Integration expanded the functions available on a chip and supported hardware description and synthesis. Pipelines, caches, and out-of-order execution exploited more single-core parallelism, with power and complexity costs.

Multicore processors, hardware multithreading, SIMD, and GPUs made parallelism increasingly important to mainstream software. Compilers, runtimes, and operating systems must expose or schedule useful work.

Historical comparisons confound technology, workload, and implementation unless their boundaries are stated.

A technology can enable a faster program without itself proving an application speedup.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter review

- The ISA defines an interface; different microarchitectures can preserve results while taking different times.
- CPU time depends jointly on dynamic instruction count, mean CPI, and effective frequency for the same interval.
- State the baseline, units and workload; measure latency and throughput separately.
- Amdahl’ s law uses baseline time fractions, and the fixed part limits total speedup.

References and attribution

Lin Cing, handwritten computer organization notes. Patterson and Hennessy, Computer Organization and Design, revised fourth edition, Chapter 1. CSO course slides, 2026, Chapter 1.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study